

敖丙知识点总结：<https://www.processon.com/view/5e86b713e4b0bf3ebcf4e376#map>

冰河并发脑图：https://cdn.jsdelivr.net/gh/sunshinelyz/binghe_resources@master/binghe/HighConcurrency/Java-Concurrent.5c7knrwgqmtc.svg

SpringBoot 流程图：

<https://www.processon.com/view/link/59812124e4b0de2518b32b6e>

排序

排序算法	平均时间复杂度	最好情况	最坏情况	空间复杂度	排序方式	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	In-place	不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
希尔排序	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(1)$	In-place	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Out-place	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	In-place	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	In-place	不稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Out-place	稳定
桶排序	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Out-place	稳定
基数排序	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	$O(n + k)$	Out-place	稳定

Java基础

		包外	子类	包内	类内
1					
2	public	yes	yes	yes	yes
3	protected	no	yes	yes	yes
4	default	no	no	yes	yes
5	private	no	no	no	yes

面向对象特性：封装、继承、多态

多态体现方式：接口、重写、重载

重载：

参数类型不同、方法名相同，与返回值，修饰符无关

重写:

子类的修饰符必须大于等于父类修饰符 (如: 父类用public, 子类可用private)

子类的返回值类型可以是父类的子类, 对于父类返回值是Void则无法修改返回值

子类抛出的异常必须小于父类抛出的异常

如果父类方法访问修饰符为 private/final/static 则子类就不能重写该方法, 但是被 static 修饰的方法能够被再次声明

父类的构造器不能重写

接口与抽象类

1. 接口的方法默认是 public, 所有方法在接口中不能有实现(Java 8 开始接口方法可以有默认实现), 而抽象类可以有非抽象的方法。

2. 接口中除了 static、final 变量, 不能有其他变量, 而抽象类中则不一定。

3. 一个类可以实现多个接口, 但只能实现一个抽象类。接口自己本身可以通过 extends 关键字扩展多个接口。

4. 接口方法默认修饰符是 public, 抽象方法可以有 public、protected 和 default 这些修饰符 (抽象方法就是为了被重写所以不能使用 `private` 关键字修饰!)。

5. 抽象类中可以有构造方法, 而接口不能有构造方法。

6. 从设计层面来说, 抽象是对类的抽象, 是一种模板设计, 而接口是对行为的抽象, 是一种行为的规范。

注意:

1. 在 jdk 7 或更早版本中, 接口里面只能有常量变量和抽象方法。这些接口方法必须由选择实现接口的类实现。

2. jdk 8 的时候接口可以有默认方法和静态方法功能。

3. Jdk 9 在接口中引入了私有方法和私有静态方法。

```
1  public interface InterfaceTest {
2      // 默认 public abstract修饰
3      int method();
4      // 默认 public static final修饰
5      int CONSTANT = 123;
6      default void method3() {
7          System.out.println("method3()");
8      }
9      // 默认public修饰, 且只能public修饰
10     static void test5() {
11
12     }
13     private void test7() {
14
15     }
16     private static void test8() {
17
```

```
18     }  
19 }
```

final关键字

修饰类：

类不能够被继承，类中的成员方法默认final方法

修饰方法：

子类不能重写父类的final方法，同时类中的 `private` 方法 被隐式指定为 `final` 修饰

修饰变量：

可以修饰成员变量、局部变量，表示该变量不能够重新赋值（对于修饰的引用，引用对象中的变量是可以修改的），

成员变量在定义时或则类的构造方法中赋值，而局部变量只能在定义时赋值

内存语义：

1. 在构造函数中对一个final域的写入，与随后把这个被构造对象的引用赋值给一个引用变量，这两个操作不能重排序，
说白了，就是在 `Obj o = new Obj();` 时，这里的赋值不能与构造函数中final域赋值重排序
2. 初次读对象引用与初次读该对象包含的final域，这两个操作之间不能重排序
3. 写final域重排序规则：禁止把final域的写 重排序到构造函数之外，编译器会在final域的写之后，构造函数return之前，插入一个StoreStore屏障，这个屏障禁止处理器吧final域的写重排序到构造函数之外

CompletableFuture

限流算法

常用的限流算法有两种：

tencent 漏桶算法和令牌桶算法。

<https://www.cnblogs.com/xuwc/p/9123078.html>

操作系统

操作系统的内存管理机制了解吗？内存管理有哪几种方式？

2. 分页机制和分段机制有哪些共同点和区别呢？
3. 逻辑地址和物理地址
4. 进程和线程的区别

产生死锁的四个必要条件：

- (1) 互斥条件：一个资源每次只能被一个进程使用。
- (2) 请求与保持条件：一个进程因请求资源而阻塞时，对已获得的资源保持不放。
- (3) 不剥夺条件：进程已获得的资源，在未使用完之前，不能强行剥夺。
- (4) 循环等待条件：若干进程之间形成一种头尾相接的循环等待资源关系。

Java线程的通信方式

1. volatile
2. 等待/通知机制
3. join方式
4. threadLocal

大端字节序：高位字节在前，低位字节在后，这是人类读写数值的方法

```
1  存储123
2  addr: 0x1000  0x1004  0x1008
3  data:  1      2      3
```

TCP、UDP区别

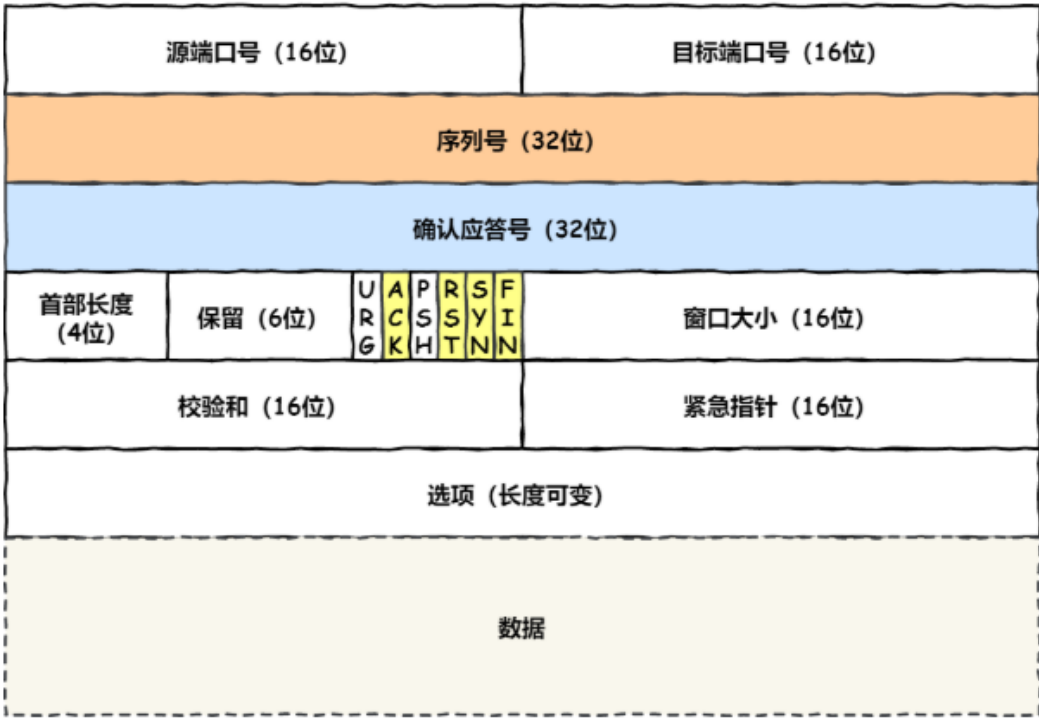
TCP如何保证传输的可靠性:

校验和，序列号和确认应答号，重传，流量控制，拥塞控制

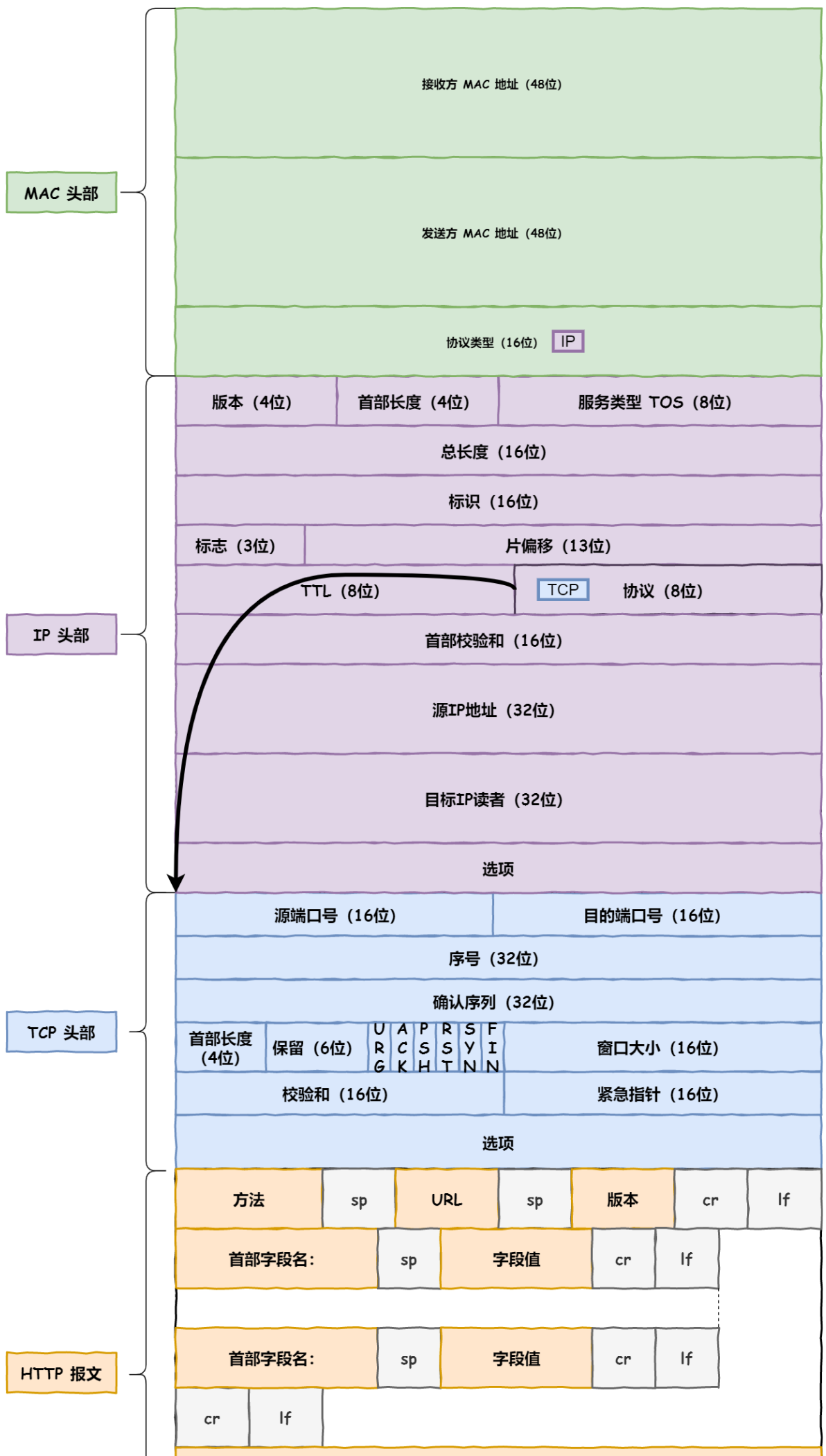
1. 应用数据被分割成 TCP 认为最适合发送的数据块。
2. TCP 给发送的每一个包进行编号，接收方对数据包进行排序，把有序数据传送给应用层。
3. **校验和：** TCP 将保持它首部和数据的校验和。这是一个端到端的校验和，目的是检测数据在传输过程中的任何变化。如果收到段的校验和有差错，TCP 将丢弃这个报文段和不确认收到此报文段。
4. TCP 的接收端会丢弃重复的数据。
5. **流量控制：** TCP 连接的每一方都有固定大小的缓冲空间，TCP 的接收端只允许发送端发送接收端缓冲区能接纳的数据。当接收方来不及处理发送方的数据，能提示发送方降低发送的速率，防止包丢失。TCP 使用的流量控制协议是可变大小的滑动窗口协议。（TCP 利用滑动窗口实现流量控制）
6. **拥塞控制：** 当网络拥塞时，减少数据的发送。
7. **ARQ 协议：** 也是为了实现可靠传输的，它的基本原理就是每发完一个分组就停止发送，等待对方确认。在收到确认后再发下一个分组。
8. **超时重传：** 当 TCP 发出一个段后，它启动一个定时器，等待目的端确认收到这个报文段。如果不能及时收到一个确认，将重发这个报文段。

网络头部信息

TCP 头部格式



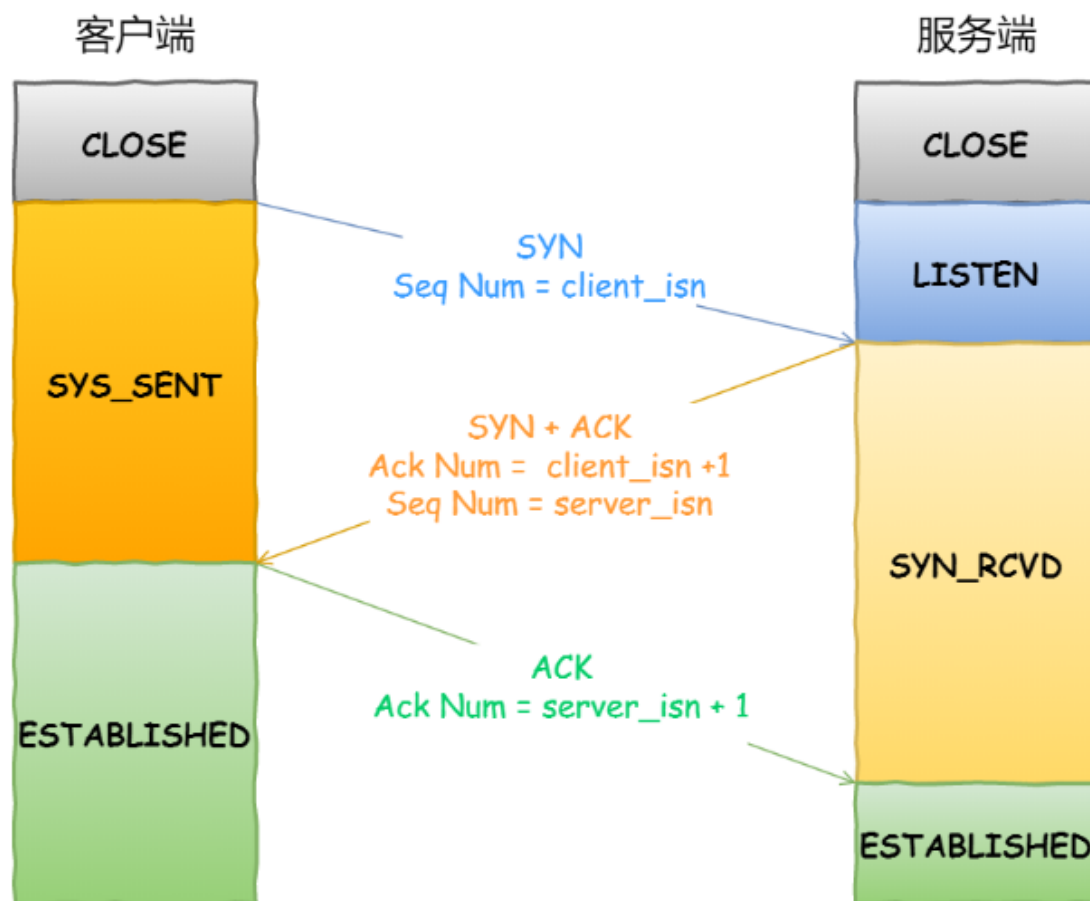
综合图：

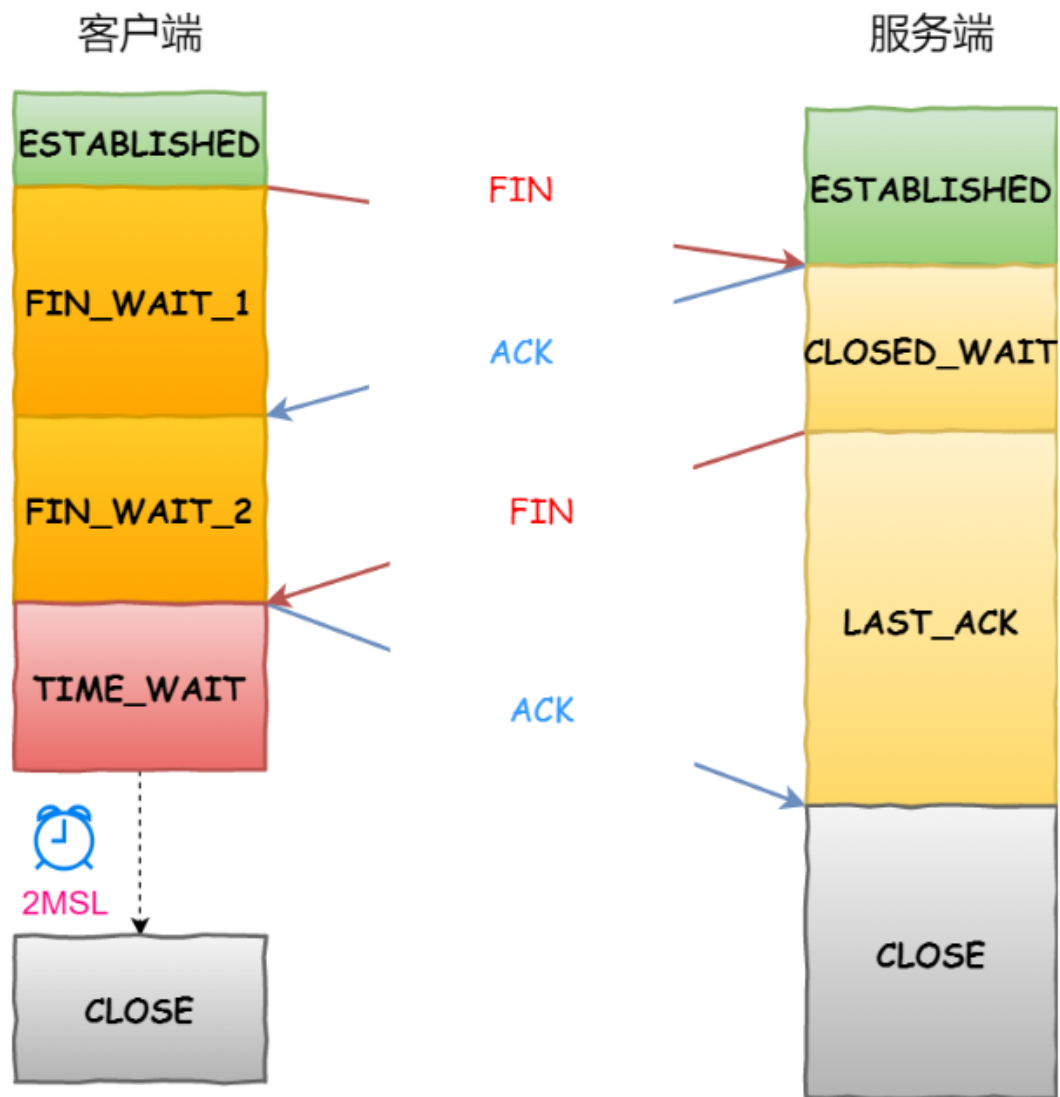


数据

建立连接

<https://mp.weixin.qq.com/s/-Sdg0Abp-CjUhlyJAiK8BA>





https://mp.weixin.qq.com/s/8t_KFtrrBkFyZKPJg_y6pw

进程、线程、协程：

<https://blog.csdn.net/daaikuaichuan/article/details/82951084>

fail-fast和fast-safe：

fast-fail：一般在迭代集合时发现该集合的结构被改变后，就有可能触发fast-fail，抛出ConcurrentModificationException

HTTP：

深拷贝、浅拷贝：

归并、快排、两个栈实现队列、最近最少使用（LRU）、打印ABC、按K位反转链表、DCL、最大heap、生成者消费者、BlockQueue、回文链表

项目系统架构图

Netty：

<https://mp.weixin.qq.com/s/I9PGsWo7-ykGf2diKklGtA>

1. Netty的线程模型基于主从多Reactor模型。通常由一个线程负责处理OP_ACCEPT事件，拥有 CPU 核数的两倍的IO线程处理读写事件。
2. 一个通道的IO操作会绑定在一个IO线程中，而一个IO线程可以注册多个通道。
3. 在一个网络通信中通常会包含网络数据读写，编码、解码、业务处理。默认情况下编码、解码等操作会在IO线程中运行，但也可以指定其他线程池。
4. 通常业务处理会单独开启业务线程池，但也可以进一步细化，例如心跳包可以直接在IO线程中处理，而需要再转发给业务线程池，避免线程切换。
5. 在一个IO线程中所有通道的事件是**串行处理**的。

selector: 线程循环询问是否准备好数据

epoll: 准备好数据后通知server处理

spring

SSM 面试题: https://blog.csdn.net/qz_33323054/article/details/104772408

Spring源码: https://blog.csdn.net/weixin_44313584/article/details/114785885

FactoryBean基于BeanFactory，FactoryBean是一个能生产或者修饰对象生成的工厂Bean；

FactoryBean

1. 创建对象时需要依赖于 别的接口 监听数据, 推送数据过来的接口 做一些数据处理时, 可以使用 FactoryBean.
- 2.
3. 2. 需要搭配其他接口协作, 依赖于Spring的某个生命周期内, 某个时间节点 来生成对象

Spring Boot

<https://blog.csdn.net/zwx900102/article/details/115206499>

自定义starter

MySQL

- SQL 等执行过程分为两类，一类对于查询等过程如下：权限校验---》查询缓存---》分析器---》优化器---》权限校验---》执行器---》引擎
- 对于更新等语句执行流程如下：分析器----》权限校验----》执行器---》引擎---redo log prepare----》binlog---》redo log commit

索引失效原因:

1. 条件中使用or

2. 对于多列索引，不是使用的第一部分，则不会使用索引
3. like查询是以%开头
4. 如果列类型是字符串，那一定要在条件中将数据使用引号引用起来,否则不使用索引
5. 如果mysql估计使用全表扫描要比使用索引快,则不使用索引

xskj 论坛

项目难点:

技术难点:

聊天功能:

Ajax 轮询:

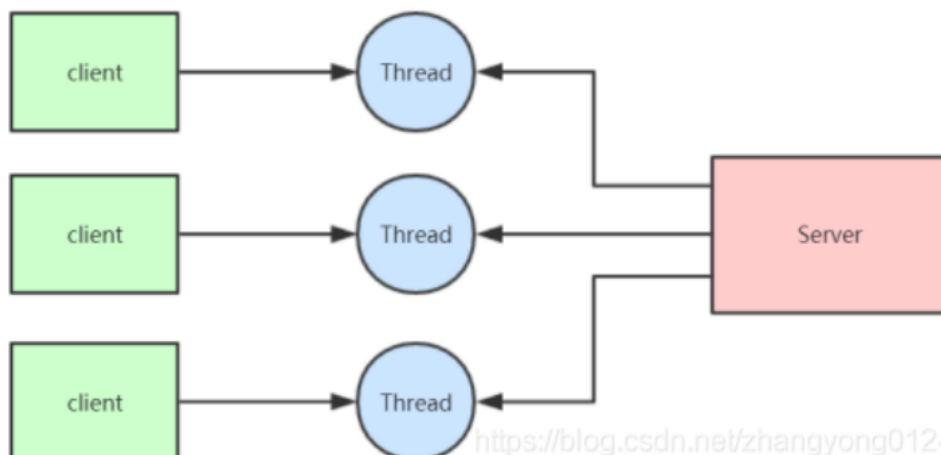
最初想法：使用ajax请求，以轮询的方式去查看是否有消息，发送消息时将消息以对象的形式 (sendId, receld, msg, state) 的形式存入数据库(MySQL, Redis) 中，当对方取出消息后，修改该数据的标识状态

这种方式实时性较差，需要定时向服务器发送请求，如果没有数据发送，将会做大量无效的轮询，给服务器和数据库巨大压力

BIO

同步阻塞模型，每一个client连接都会创建一个线程，当用户数太多后，将会导致服务器线程创建过多而导致性能急剧降低，从而导致宕机

实现思路：将所有的socket连接对象，作为一个单独的线程，每个线程监听是否有读写任务，同时将<id, socket> 存入Map对象中，方便指定传输对象



<https://blog.csdn.net/zhangyong0124/>

NIO

非阻塞IO模型，每一个连接对应一个channel，将其注册到selector中，由selector来监听(epoll)那些channel 有事件发生，这样让一个线程处理多个连接，减小服务器的压力。

Netty:

Netty基于NIO的一种通信框架，对NIO进行了大量的封装，降低了NIO编写的复杂性，

```
1  1. 每个用户登录时，都会创建一个websocket连接，40s(boss)发送一个心跳
2
3      消息状态： 已读，未读
4  2. 发送消息时，
5      数据默认标识未读状态
6      判断对方是否在线，如果没有在线，消息写入数据库
7      对方在线，那么将数据发送给对方，对方会收到提示，进入聊天界面后，修改数据库，将当天聊天消息全
   部设置为已读
8
9  3. 第一次登录账号时，首先会建立webSocket连接， 服务器检测是否有未读的消息，如果有发送到浏览
   器，浏览器会收到消息提示，前端用数组来缓存 收到的消息（如最新的10条）
10
11 4. 当用户已经登录后，在正常浏览网页过程中，收到的消息都是由webSocket维护，当收到消息后，
   webSocket将消息存入缓存中，当用户打开聊天界面，就会显示这些聊天记录，同时会发送请求标记对方用户
   发送给自己的消息已经全部已读。
12
13 只有第一次登录，才会检查数据库是否有未读的消息，这里的未读主要是发送到浏览器记录，后续的都是在这
   个未读的消息上追加
```

```
1  进入聊天界面，先发送ws请求，表示向webSocket握手
2  握手后， socket.onopen()方法中，发送register向netty注册自己的ChannelHandlerContext到
   Map<userId, ChannelHandlerContext>中
3  发送消息：
4  前端页面采用json封装发送的数据，使用socket.send(obj)
5  后端接受消息：
6
7  自定义Handler继承SimpleChannelInboundHandler
8
9  HttpRequestHandler#channelRead0() 判断消息类型，连接请求，还是文本类型
10
11 WebSocketServerHandler#channelRead0() -> handlerWebSocketFrame() 这里要判断是否是
   关闭，还是文本，还是心跳检测，如果是文本，获取目标用户toUserId，在map中寻找是否有目标用户所对
   应的ChannelHandlerContext，如果有则使用ctx.channel().writeAndFlush(msg)发送消息
12
13
14 心跳检测：
15 // 用于后端检测是否有读写事件发送
16 addLast(new IdleStateHandler())
17 // 前端心跳： 定时向后端发送一个数据，用来保持长连接
18
19
20 消息可以数据库存一份，以便查看聊天记录
21 如果对方没有在线，发送的数据暂时存入一个未发送成功的数据库，待对方登录后，检测这个数据库，如果有
   没有接受的消息，直接使用websocket 推送
```

每个连接对应一个ChannelHandlerContext

Session共享

由于本项目采用的分布式开发，为了保证多个模块之间保存用户身份信息，最开始采用的是session共享的方式，登录成功后，将生成一个sessionId，将它与User信息一起存入redis，同时在浏览器存下，下次访问时带上该session，在需要验证用户身份的模块直接去redis中取看是否存在session，如果不存在说明没有登录，

这种方式在请求时会经常向redis服务器发送请求，效率降低，

由于某些业务会经常判断用户身份，为了避免重复的解析用户信息，某些条件下会经常判断该用户身份信息，是否死管理员身份啥的，由于每个请求tomcat都会生成一个线程，因此采用了ThreadLocal来保存当前用户信息，在使用当前请求结束时，remove掉，防止内存泄漏。

后面通过学习，发现JWT 通讯更加方便，服务器无需保存信息，只需要在生成时返回给浏览器，浏览器下次请求携带过来，服务器作下简单的验证就可以了。

RabbitMQ:

为了提高系统的响应速度，在账号注册时，发送邮件验证码 交给 MQ，让注册服务去MQ中取消息，进而触发真正的消息验证服务

文章模块、问答模块、发现模块、吐槽模块

将每个模块分别注册到Eureka上，通过zuul网关进行拦截请求

FeignClient用来在一个模块直接调用另一个模块功能

取消登录验证码，

- 文章模块：发布文章，对文章进行评论（评论信息放到mongodb），评论信息的点赞（每天只能点一次赞，使用redis来记录是否已经点过赞，过期时间设置为当前时间到0点的差）
 - 文章评论点赞后需要刷新
- 问答模块： problem绑定标签，reply 绑定problem，
 - 最新回答： 查询replytime字段
 - 最热回答： 根据problem的reply数量
 - 等待回答： 更加create时间同时reply为0，没有人回答过

前端： nuxt（vue实现的快速开发web工具，用于将页面进行分离，同时使用pages的目录结构来代替路由）、vuex（用于保存页面的状态信息，方便其他页面使用，this.\$store）、vue、mavon-editor

对axios创建request实例，添加请求拦截器，在每次请求发送时header加上Authorization字段

问答，时间显示问题，登录验证码生成，报错解决

JWT安全性：

设置过期时间一周，隔一段时间重新生成令牌

注册功能

先使用Redis验证验证码是否正确，未过期，生成用户对象，密码采用BCryptPasswordEncoder类(SHA-256+随机盐+密钥，每次加密的密文都不一样)进行加密，插入MySQL，

登录功能

验证用户密码，密码通过matches方法判断，输入的密码与数据库中的密码是否能匹配上，

- 1 \$2a\$10\$/GtzWl/YaMWtYs.v6eJazu.4QVkJAoCYWWVRa1Rbk1xLiUMQBWZpa，根据他的源码可以看到这个加密串里面分为了几部分：\$2a 固定的前缀 \$10\$ 数字10 是一个长度值，默认是10，可以new 对象的时候指定，/GtzWl/YaMWtYs. 是真正的生成的盐值，v6eJazu.4QVkJAoCYWWVRa1Rbk1xLiUMQBWZpa 是盐值和密码 根据随机数 然后根据他内置的一个base64算法生成的一个字符串，然后将这几部分拼接起来就是你得到的加密串
- 2
- 3 匹配过程：取出了用户密码 加密串里面的盐值，然后再拼接前面部分， 拼接传入的密码进行生成密码值这一步操作，用它自己的一个base64算法得到一个加密串，将其与旧的加密串比对

匹配成功后，生成token

- 1 token优点：
- 2 **支持跨域访问**
- 3 无状态(也称：服务端可扩展行)
- 4 **更适用CDN**：
- 5 去耦
- 6 CSRF
- 7 性能

这里采用JWT 的token认证技术，由 头部、载荷与签名 组成

- 1 头部(header)： 类型，签名所用算法
- 2 载荷(payload)：存放jwt签发的信息，用户，jwt签发时间，过期时间
- 3 签名(signature)：
- 4 base64 (base64(header)+base64(payload)+secret)
- 5
- 6 jwt: base64(header).base64(payload).signature

采用JWT生成token后返回给client，client可以将token存入cookie或storage，在下次请求时代上token即可，同时server不需要保存token 信息，只需要通过服务端的secret验证token，secret 不能泄漏，一旦泄漏，client就可以自我签发jwt

解决过期问题：

在每次检验token时，判断过期时间是否快要过期，设置一个阈值，如果满足重新设置一个新的token

相关技术

Redis

首页显示热门的文章，并对热门的文章存入Redis中，提高响应速度，热门文章保存时间一天，一天后自动重新写入新文章（根据当天文章访问量来区分是否为热门文章，排名前10）

MongoDB

端口：27017

spring data mongodb 面向文档的数据库，存放文章评论信息，

- (1) 数据量大
- (2) 写入操作频繁（读写都很频繁）
- (3) 价值较低

MongoDB	关系型数据库Mysql
数据库(databases)	数据库(databases)
集合(collections)	表(table)
文档(document)	行(row)

ES

Relational DB	Elasticsearch
数据库(database)	索引(indices)
表(tables)	types
行(rows)	documents
字段(columns)	fields

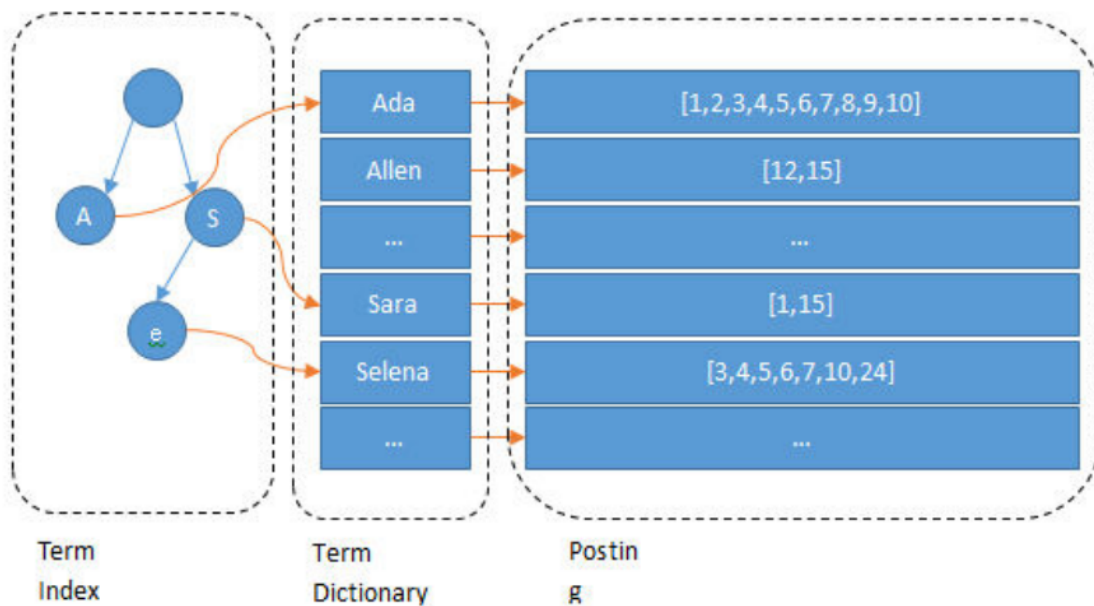
实时的分布式搜索和分析引擎。它可以帮助你前所未有的速度去处理大规模数据。ElasticSearch是一个基于Lucene的搜索服务器。它提供了一个分布式多用户能力的全文搜索引擎

elasticsearch 在后台把每个**索引划分成多个分片**，每分片可以在集群中的不同服务器间迁移
一个人就是一个集群！默认的集群名称就是 elasticsearch

<https://blog.csdn.net/iyoly/article/details/101926831>

https://blog.csdn.net/weixin_44588495/article/details/105340593

为什么 Elasticsearch/Lucene 检索可以比 mysql 快了。



首先es对每个field都建立 倒排索引，在搜索数据时通过term dictionary拿到对应的id数组，在id数组中寻找相应的值即可，这里并不是简单直接在term dictionary中查找，es会对term dictionary 使用trie算法构建一个查找树，通过trie树能够迅速定位到 term dictionary 的某个offset，从而迅速找到相应的id数组，es采用 FST压缩算法是term index 的大小是所有的term 的几十分之一，降低了缓存term index的内存大小

相对于MySQL来说：

MySQL 只有 term dictionary 这一层，是以 b-tree 排序的方式存储在磁盘上的。检索一个 term 需要若干次的 random access 的磁盘操作。而 Lucene 在 term dictionary 的基础上添加了 term index 来加速检索，term index 以树的形式缓存在内存中。从 term index 查到对应的 term dictionary 的 block 位置之后，再去磁盘上找 term，大大减少了磁盘的 random access 次数。

FST(finite-state transducer)

实际上Lucene采用的是一种变种的trie树，

相对HashMap来说，fst更加省内存

ES分片技术

ES的架构：Cluster - Node - ES Index - Shard(Lucene Index) - Segment

首先最大的概念是Cluster集群，ES是天然的分布式架构，由多台机器组成。每一台机器称为一个Node节点，节点内保存着多个Index的部分Shard。每个Shard实际上是一个Lucene Index，里面被分成了数个Segment，而Segment即为我们存储数据的最小单位。

因为不同的Shard分布在不同的Node，因此可以多进程并发的去搜索各个Shard，提高了吞吐量和性能

另外，我们的Node可分为 **主节点**（维护索引元数据、负责切换Shard主从身份），数据节点和客户端节点（请求的分发与结果的汇总），此外还有 **协调节点** 的概念，负责转发索引请求到对应的Shard。每个节点都知道集群中 **任意文档** 的位置，因此 **每个节点** 都可作为协调节点，即每个节点都可响应客户端的请求，这也在客观上提高了系统的并发能力。协调节点会使用负载均衡的算法将查询请求分发到主从Shard上，来进行读请求的负载均衡。

ES 采用乐观并发控制来保证数据的正确性，同时，ES本身基于的Lucene库在底层将数据分为segment，几乎避免了锁的出现，大大的提升了读写效率。

项目中如何使用

1. 你在项目中如何开发搜索模块

Spring Data Elasticsearch操作elasticsearch

2. 你如何实现数据库与索引库的同步？

logstash

配置数据输入，输出地址

3. 你如何实现索引库的分词

IK 两种算法 最少切分 最细切分(词语，单个都会出现)

Elasticsearch与Solr的比较

1. 当单纯的对已有数据进行搜索时，Solr更快。
2. 当实时建立索引时，Solr会产生io阻塞，查询性能较差，Elasticsearch具有明显的优势。
3. 随着数据量的增加，Solr的搜索效率会变得更低，而Elasticsearch却没有明显的变化。

综上所述，Solr的架构不适合实时搜索的应用。

RabbitMQ

应用耦合，异步消息，流量削锋，消息通讯

用户注册账号：

用户模块生成验证码，将验证码和邮箱地址信息存入Redis定时，同时发送到MQ中，邮件服务器从MQ中拿到消息，发送相应的邮件

问题

1. 项目中哪部分业务用到消息队列

用户注册发送短信验证码、商品下单后计算积分等

2. 项目中使用哪种消息队列？

RabbitMQ、ActiveMQ

3. RabbitMQ 有哪几种模式

6种工作模式

1、Work queues

多个消费端消费同一个队列中的消息，队列采用轮询的方式将消息是平均发送给消费者

2、Publish/subscribe

相对于Work queues模式，该模式多了一个交换机，生产端先把消息发送到交换机，再由交换机把消息发送到绑定的队列中，每个绑定的队列都能收到由生产端发送的消息

3、Routing

Routing 模式又称路由模式，该种模式除了要绑定交换机外，发消息的时候还要制定routing key

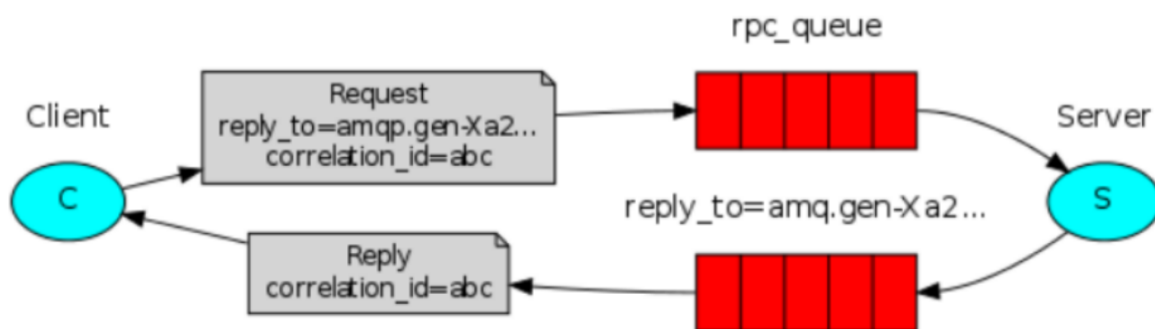
4、Topics

Topics 模式和Routing 路由模式最大的区别就是，Topics 模式发送消息和消费消息的时候是通过通配符去进行匹配的

5、Header 模式

header模式取消routingkey，使用header中的 key/value（键值对）匹配队列。

6、RPC



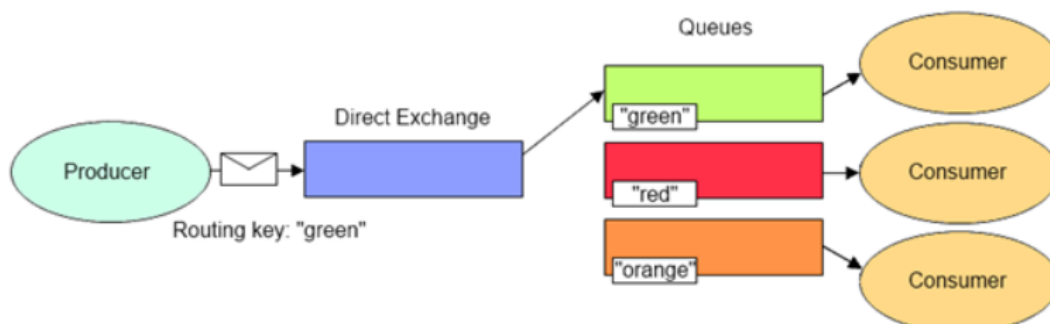
MQ可以实现RPC的异步调用，基于Direct交换机实现

4种交换机类型：

Exchange类型

交换器说到底是一个名称与队列绑定的列表。当消息发布到交换器时，实际上是由你所连接的信道，将消息路由键同交换器上绑定的列表进行比较，最后路由消息。

Direct Exchange直连交换机



直连型交换机（direct exchange）是根据消息携带的路由键（routing key）将消息投递给对应队列的。设置 Exchange 和 Queue 的 Binding 时需指定 RoutingKey（一般为 Queue Name），发消息时也指定一样的 RoutingKey，消息就会被路由到对应的Queue。直连交换机用来处理消息的单播路由（unicast routing，尽管它也可以处理多播路由）（一对一模式）

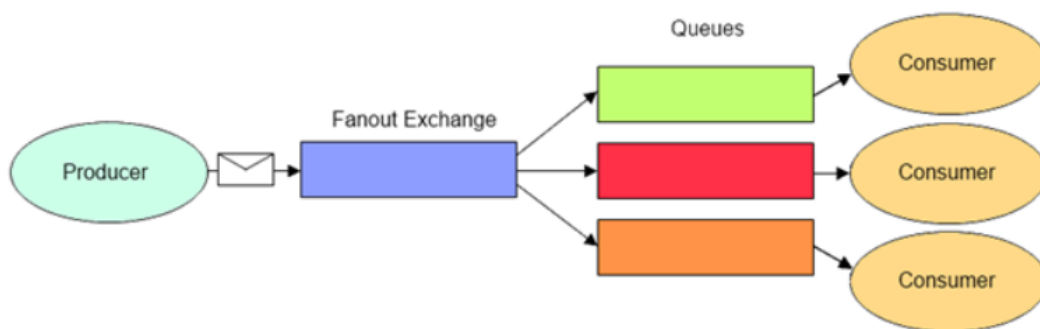
工作机制：

1. 将一个队列绑定到某个交换机上，同时赋予该绑定一个路由键（routing key）。
2. 当一个携带着路由键为green的消息被发送给直连交换机时，交换机会把它路由给绑定值同样为green的队列。

说明：

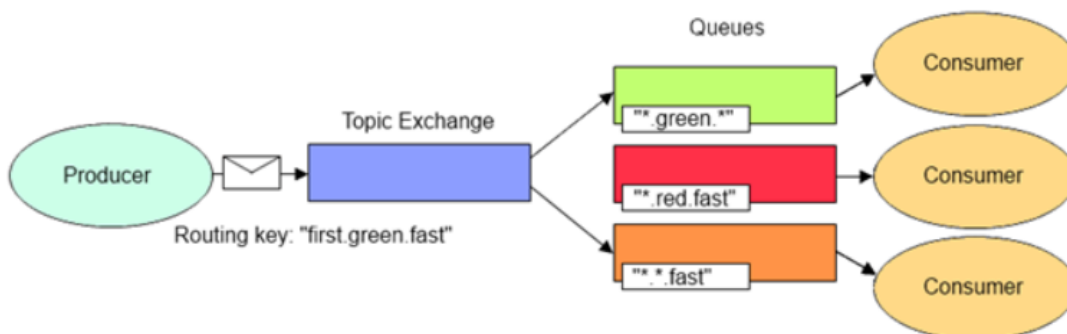
- 消息传递时需要一个“RouteKey”，可以简单的理解为要发送到的队列名字。
- 这种模式需要提前将Exchange与Queue进行绑定，一个Exchange可以绑定多个Queue，一个Queue可以同多个Exchange进行绑定。在绑定Queue的同时，需要绑定routingKey。一般Queue和routingKey的名字一样。
- 如果接受到消息的Exchange没有与任何Queue绑定，则消息会被抛弃。
- 一般情况下，可以直接使用Default Exchange，因为不需要将Exchange进行任何绑定(binding)操作。
- 如果使用Default Exchange，vhost中不存在RouteKey中指定的队列名，则该消息会被抛弃。

Fanout Exchange扇出(群发、广播)交换机



扇出交换机（fanout exchange）将消息路由给绑定到它身上的所有队列，而不理会绑定的路由键

Topic Exchange话题交换机



主题交换机（topic exchanges）通过对消息的路由键和队列到交换机的绑定模式之间的 **匹配**，将消息路由给一个或多个队列。主题交换机经常用来实现各种分发/订阅模式及其变种。主题交换机通常用来实现消息的多播路由（multicast routing）。

Topic Exchange 配置上和 Direct Exchange 类似，也需要通过 **RoutingKey** 来路由消息，区别在于 Direct Exchange 对 RoutingKey 是精确匹配，而 Topic Exchange 支持模糊匹配。

关于通配符，分别支持 ***** 和 **#** 通配符，***** 表示匹配一个单词，**#** 则表示匹配一个或者多个单词。因此first.# 能够匹配到 first.green或first.green.fast，但是 first.* 只会匹配到 first.green，不能匹配 first.green.fast。

Headers exchange

有时消息的路由操作会涉及到多个属性，此时使用消息头就比用路由键更容易表达，头交换机（headers exchange）就是为此而生的。头交换机使用多个消息属性来代替路由键建立路由规则。通过判断消息头的值能否与指定的绑定相匹配来确立路由规则。

Headers Exchange 会忽略 RoutingKey 而根据消息中的 Headers 和创建绑定关系时指定的 Arguments 来匹配决定路由到哪些 Queue。

Headers Exchange 的性能比较差，而且 Direct Exchange 完全可以代替它，所以不建议使用。

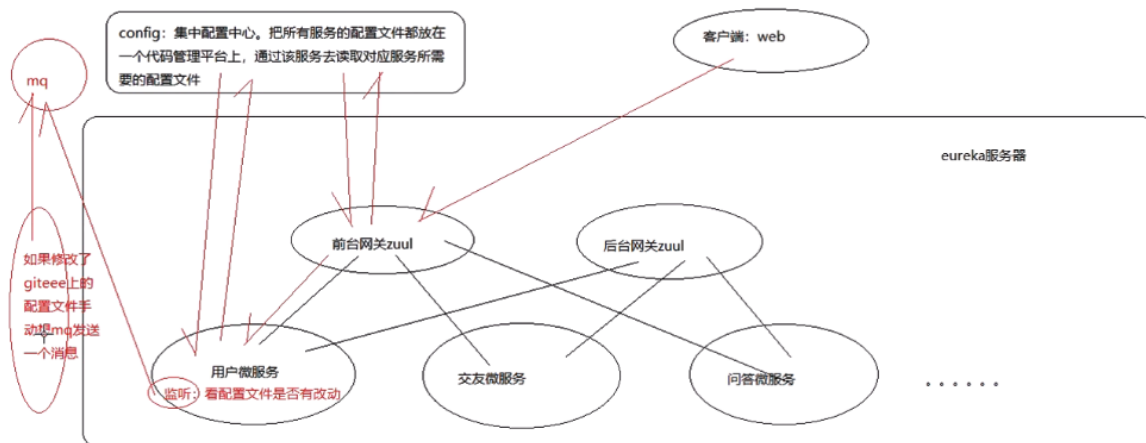
直接（直连）模式，分列（扇出、群发、广播）模式，主题模式，headers

每个登录的用户对应一个ChannelHandlerContext

SpringCloud

- 服务发现——Netflix Eureka
 - 将每个微服务发布到eureka
- 服务调用——Netflix Feign
 - 实现微服务各个模块之间相互调用
 - 已经依赖负载均衡组件ribbon，（轮询，随机，权重）
- 熔断器——Netflix Hystrix
 - 防止服务级联失败
- 服务网关——Netflix Zuul
 - 作为网关，所有的请求都先到这，可以进行身份验证
 - （1）微服务工程统一入口，方便前端调用
 - （2）集中处理权限问题等
- 分布式配置——Spring Cloud Config
 - 为了方便服务配置文件统一管理，实时更新，所以需要分布式配置中心组件
- 消息总线 —— Spring Cloud Bus
 - 不重启微服务的情况下更新配置

架构图：



JWT

一个JWT实际上就是一个字符串，它由三部分组成，头部、载荷与签名。

头部 (Header)

头部用于描述关于该JWT的最基本的信息，例如其类型以及签名所用的算法等。这也可以被表示成一个JSON对象。

```
1 {"type": "JWT", "alg": "HS256"}
```

- type: 是类型。
- alg: 签名的算法，这里使用的算法是HS256算法

我们对头部的json字符串进行BASE64编码（网上有很多在线编码的网站），编码后的字符串如下：

```
1 eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9
```

小知识：Base64是一种基于64个可打印字符来表示二进制数据的表示方法。由于 2^6 等于64，所以每6个比特为一个单元，对应某个可打印字符。三个字节有24个比特，对应于4个Base64单元，即3个字节需要用4个可打印字符来表示。JDK 中提供了非常方便的 Base64Encoder 和 Base64Decoder，用它们可以非常方便的完成基于 Base64 的编码和解码。

载荷 (payload)

载荷就是存放有效信息的地方。这个名字像是特指飞机上承载的货品，这些有效信息包含三个部分：

(1) 标准中注册的声明（建议但不强制使用）

- 1 iss: jwt签发者
- 2 sub: jwt所面向的用户
- 3 aud: 接收jwt的一方
- 4 exp: jwt的过期时间，这个过期时间必须要大于签发时间
- 5 nbf: 定义在什么时间之前，该jwt都是不可用的。
- 6 iat: jwt的签发时间
- 7 jti: jwt的唯一身份标识，主要用来作为一次性token,从而回避重放攻击。

(2) 公共的声明

公共的声明可以添加任何的信息，一般添加用户的相关信息或其他业务需要的必要信息.但不建议添加敏感信息，因为该部分在客户端可解密。

(3) 私有的声明

私有声明是提供者和消费者所共同定义的声明，一般不建议存放敏感信息，因为base64是对称解密的，意味着该部分信息可以归类为明文信息。

这个指的就是自定义的claim。比如前面那个结构举例中的admin和name都属于自定的claim。这些claim跟JWT标准规定的claim区别在于：JWT规定的claim，JWT的接收方在拿到JWT之后，都知道怎么对这些标准的claim进行验证(还不知道是否能够验证)；而private claims不会验证，除非明确告诉接收方要对这些claim进行验证以及规则才行。

定义一个payload:

```
1 {"sub": "1234567890", "name": "John Doe", "admin": true}
```

其中sub是标准的声明，name和admin是自定义的声明（公共的或私有的）

然后将其进行base64编码，得到Jwt的第二部分：

```
1 eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWYWRtaW4iOnRydWV9
```

提示：声明中不要放一些敏感信息。

签证、签名 (signature)

jwt的第三部分是一个签证信息，这个签证信息由三部分组成：

- 1) header (base64后的)
- 2) payload (base64后的)
- 3) secret (盐，一定要保密)

这个部分需要base64加密后的header和base64加密后的payload使用.连接组成的字符串，然后通过header中声明的加密方式进行加盐secret组合加密，然后就构成了jwt的第三部分：

```
1 TJVA950rM7E2cBab30RMHrHDcEfxjoYZgeFONFh7HgQ
```

将这三部分用.连接成一个完整的字符串,构成了最终的jwt:

```
1 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWYWRtaW4iOnRydWV9.TJVA950rM7E2cBab30RMHrHDcEfxjoYZgeFONFh7HgQ
```

注意：secret是保存在服务器端的，jwt的签发生成也是在服务器端的，secret就是用来进行jwt的签发和jwt的验证，所以，它就是你服务端的私钥，在任何场景都不应该流露出去。一旦客户端得知这个secret,那就意味着客户端是可以自我签发jwt了。

简历

511602199703047399

专业技能

- 掌握Java基础语法，熟悉面向对象、反射、集合类、I/O、多线程、设计模式等内容

- 对JVM有初步的理解，包括内存模型、内存结构、垃圾回收算法、垃圾回收机制
- 熟悉Spring、SpringMVC、Mybatis、SpringBoot、Netty等主流开源框架
- 熟悉MySQL基本CRUD，了解事务、索引、日志、缓存、锁、MVCC等
- 熟悉Redis常用数据类型，了解底层数据结构、数据持久化策略、内存淘汰策略等
- 熟悉常用数据结构：数组、链表、栈、队列、堆、二叉树等
- 熟悉常用算法：排序、哈希、DFS、BFS、二分、动态规划、并查集等
- 熟悉计算机网络基础知识，熟悉TCP/IP 四层模型、HTTP等基本网络协议
- 了解Linux基本命令、Shell编程、以及Docker容器、Nginx
- 熟练使用IDEA、MAVEN、Git等开发工具

项目经验

XS交流平台

时间： 2021.1 — 2021.3

本项目采用前后端分离开发，用作知识共享交流。主要功能有：发表技术 文章,评论文章,知识问答,文章点赞、实时沟通等

主要开发模块：

用户模块： 包含登录注册功能，使用JWT来保存会话信息

文章模块： 登录用户可以发表文章，对文章进行评论，对文章评论进行点赞，未登录用户只能预览文章

在线交流： 使用Netty来实现用户之间的实时聊天功能，以及消息推送功能

发布上线： 对前端项目打包上传服务器，通过配置Nginx来让外界访问，后端用Docker部署

<https://gitee.com/xiaoye2018/xsjl/>

https://blog.csdn.net/qq_42641075

通讯

1. 实时通讯

- Ajax短轮询
 - ajax完全无状态，每次请求后，都要断开连接
 - 不停的向服务器发送请求，询问是否有消息，浪费带宽
- Ajax长轮询
 - 使用ajax实现的长轮询，服务器向客户端以流 (stream) 的形式发送数据，类似于视频传输推送数据到客户端，SSE就是利用这种机制
 - 相较于短轮询，长轮询较少的请求发送的次数
- 短轮询、长轮询异同：
 - 相同点：当server 的数据不可达时，基于http长轮询和短轮询 的http请求，都会 停留一段时间；
 - 不同点：http长轮询是在服务器端的停留，而http 短轮询是在 浏览器端的停留
 - 由于服务器所能承载的TCP连接是有限的，这两种都容易让服务器TCP连接缓冲区溢出
- websocket
 - 基于h5协议，长连接，只需要一次握手，整个通信都建立在一个连接上

- websocket支持全双工模式，而SSE属于单向模式，只能服务器向客户端推送消息，因为流信息本质是下载
- 支持服务器推送

项目中的通讯：

点击对方头像后，进入聊天界面，才可以发送消息

如果对方ChannelHandlerContext

实习内容

内江市中医医院

职位：程序员

部门：信息科

时间： 7.1 - 7.30

工作描述：

负责医院官方网站建设与维护，包括前端界面修正，以及后端一些功能调整

前端添加打印 按钮， 后端记录下打印次数

负责医院网站建设与维护， 处理医院常规的网络问题

跟专业关联性不大，已离职

没干什么事， 业余时间就学点自己的东西

自我评价

乐观开朗，适应能力强

有良好的设计和编码品味，热爱编程

喜欢阅读源码，看技术博客，分享技术知识

场景题

- 1 首先一道很简单的智力题，三人过桥，桥最多同时两人，分别耗时1，2，5分钟，1个手电筒，问最短过桥时间
- 2 情景题：如果一个外卖配送单子要发布，现在有200个骑手都想要接这一单，如何保证只有一个骑手接到单子？
- 3 场景题：美团首页每天会从10000个商家里面推荐50个商家置顶，每个商家有一个权值，你如何来推荐？第二天怎么更新推荐的商家？
- 4 场景题：生产者消费者模式。手写一个线程安全的生产者与消费者。
- 5 场景题：微信抢红包问题，我说可以用生产者消费者模式解决，采用乐观锁，

6 说了两种方式之后，面试官继续追问：有没有不阻塞的方式，我说有，但是忘记了，面试官说你回去再看看吧

7 6 启动项目时如何实现不在链接里输入项目名就能启动？这个当时没回答上来，面试官说缺少项目经验，等你工作了两年就熟练了。

8 7 场景题：1000个任务，分给10个人做，你怎么分配，先在纸上写个最简单的版本，然后优化

9 8 怎么解决你这个系统高并发的问题？

10 我说可以用负载均衡来平衡流量，扩大服务器规模，面试官说你数据库服务器不要处理嘛，我赶紧补了一句可以用缓存

11 负载均衡怎么配置？只看过介绍，没配置过阿

12 缓存找到了数据怎么配置，找不到又怎样处理？画一下

13 9 10亿个扣扣号，每秒有10万条请求查询过来，如何处理？

14 10 规定1分钟之内只能处理1000个请求，你怎么实现，手撕代码。写好了之后，面试官一再强调一分钟是相对时间，感觉我的写法面试官并不满意，然后这个问题纠缠了很久

15 11 id全局唯一且自增，如何实现，我提到了数据库中的自增主键，他说不是数据库呢，我就说用TreeMap，有序且唯一，他说可以

16 12 如何设计一个循环队列（buffer），数组，链表。。

17 13 场景题：保证发送消息的有序性，消息处理的有序性。分布式处理的一些概念。CAS,ABA,MVCC。

18

19 14 如何把一个文件快速下发到100w个服务器（不会）

20 （面试官之后说你可以想想迅雷是怎么做到下载速度那么快的）

21 15 统计文本中出现频率最高的前五个单词

22 16 设计题，给每个组分配不同的IP段，怎么设计一种结构使的快速得知IP是哪个组的（开始想直接NAT协议搞一搞每个IP一个组的。。面试官说场景不对是给每部分的IP分组。。想用Trie树，面试官让我再优化一下。。）

23 17 让你设计一个数据库连接池你需要对数据库连接进行怎么样的封装，要设计哪些模块，怎么解决长时间无操作服务器把某连接置为不可用但是客户端还是以为是可用的问题（我说了定时任务）

24 18 多线程问题，有10个线程，每个线程作自增到10000，怎么保证不出错！）

25 19 假设一个场景，要求 stop the world时间非常短，你会怎么设计垃圾回收机制？

26 20 智力题，两个人取100个小球，每次取1个或2个，最后拿不到球的人输，如果你是先手，有什么策略可以一定赢。

27 21 3点15分时针和分针的角度是多少度？7.5度

28 22 2017的2017次方的最后一位数是什么？

29 23 八个球，有一个比较重的，给一个没有刻度的天平，怎么最快找出。

30 24 做一道题吧：有100盏灯，编号依次为1,2,3...100，电灯全部关着。现在来了100个人，第一个人把所有的灯开关按下；第二个人隔一个灯按下（2,4,6...）；第三个人每隔两个灯按下（3,6,9...）。第100个人隔99个灯按下（100），最后还有几盏灯，那几盏灯亮着？（100个人，那个应该是算完全平方数的吧，只要因数是奇数个就会亮着）

31 25 场景题，有一个文件，每行数据的类型不定、文件大小不定，请设计一个上传的方案。没答上，GG。

32 26 如何保证集群环境下抢购的并发安全？

33 27 如何设计一个定时器定时完成某个任务？

34 28 现在有很多数据（用户的订单交易数据）需要加载带内存中（不用考虑内存是否足够），使用什么数据结构来实现快速查找某一时间段的用户交易数据。

35 29 场景设计题：用户购买了某一些股票，我们如何设计一个当股票上涨一定的幅度之后给用户推送提醒。（股票数据每一分钟，都在变化，即相当于每一秒都给你发送很多数据，然后进行处理实现推送逻辑）

36 30 电商秒杀业务，如何保证不超卖，还要保证服务可用

37 31 逻辑题：6个赛道，36匹马，找出 top 1（如果是 top 6 呢）

38 32 2G 内存 1T数据 如何找出前1000大的数，我说最小堆，他说还能更快嘛，我说分治，他说怎么分治，他让我说细节（本人太菜，细节我还是没讲清楚）

39 33 火车票抢票，只有一台服务器，瞬时访问量很大，如何系统的解决？

40 34 一篇英文文献，怎么找出出现次数最多的单词？

41 35 问了一个tomcat服务器部署项目，如果在执行定时任务时候服务器启动失败了，怎么办，答：可以增加服务器，容错，那如果两台服务器都部署项目，怎么让一个去执行呢，答

面试：

心态

老子天下第一，

问题回答完，可以说，以上就是我对xxx的理解。。。

自我介绍：

技术面

您好，我叫叶强，来自四川广安，就读于一所普通本科院校，软件工程专业。

在大学期间学习过计算机网络、操作系统、数据结构、C语言、Java等课程，除了学校的课程外，自己也自学了一些其他的技术，比如SpringBoot，Linux等，同时读过Java的一些源码，也做过一些Java相关的小项目，同时自己也比较喜欢看技术博客来巩固自己的知识，同时扩宽自己的知识面，提升自己的专业水平。 我的自我介绍说完了，谢谢

HR面

您好，很荣幸有机会来参加公司的面试。我叫叶强，家乡在四川，就读于一所本科院校，专业是软件工程，明年6月份毕业。因为热爱写代码所以走上程序员这条路，所以来面试这个职位。

在大学期间学习过计算机网络、操作系统、数据结构、Java等课程，都取得了优异的成绩，平时除了学习学校安排的课程，自己也自学了很多其他的技术，

我在大学期间担任过班长，组织过一些集体活动，另外我对写代码充满热情，不喜欢半途而废，就以前写一个项目出来一个bug，一直在网上搜寻资料寻求答案，连续搜寻了几天，国内网站的大部分文章都看过了，最后在国外网站找到答案，

平时也喜欢通过跑步来放松自己，因为我相信只有一个健康的身体，才能为公司创造最大的价值。
我的自我介绍说完了，谢谢！

期望薪资

我相信公司有成熟的薪酬体制，愿意接受相应岗位的薪资福利

面试给的建议

1. 对知识总结不到位，回答容易跑题
2. 算法上的编码能力欠缺

想问的问题：

- 我的回答有什么建议吗
- 团队主要做的事情

算法：

认真审题，平台没有测试用例， 需要看清楚条件

号，下标

如，

只需

果版

本

0

那应该就只招测开了把，我这套卷，服务端&基础平台开发&3D图形算法，出的题都很简单，比较版本号、三数之和、数组右移

发表于 今天 17:29:57

举报 公告 赞(0) 回复(2)

洗衣机，太难了测开

—	1224. 最大相等频率	64	32.3%
—	920. 播放列表的数量	18	47.6%
✓	517. 超级洗衣机	44	

字节：

- 取模结果在 A~B范围内， $x \% (B - A) + A$

```
1  abcd...z -> 每个字母+k，得到的新字符串，只能出现a-z字母
2  cur = (char)((cur + k - 'a') % 26 + 97);
```

- 公共子串，？

```
1  ab
2  ac
3
4  子串：
```

- 获取最大值

```
1      1 2 5 4 3
2      5 3 2 1 1
3  +   6 5 7 5 4
4  max: 7  min: 4, 选择第二项 3 > 1
5  输出: 7 4 2(第二项)
6      1 2 5 4 3
7      5 3 2 1 1
8      1 5 7 5 2
9      4 2 2 7 1
10     3 3 2 5 1
11
12     两两组合，判断哪两项值最大，同时输出相应编号
13
14
```

- 滑动谜题

•

连连支付：

一面：

- 聊项目

- RabbitMQ:
 - 降低系统之间的耦合度，如果模块之间不存在相互调用，新增模块对其他模块影响就较小，这样系统的扩展性就更好
 - 提高系统响应速度，用户请求发送给消息队列后直接就返回了。
 - 不使用消息队列，所有用户的请求都会落到服务器上，当服务器处理不过来，将会导致服务器宕机。当使用你的消息队列，用户的请求数据直接发送给消息队列后直接返回，业务处理进程在从消息队列中去拿数据进行处理，处理完成后在异步将数据写入数据库
- ArrayList、LinkedList
- springmvc执行流程
- redis跳表（单向链表、一层一层），没说清，zset使用调表，zset有序，通过score来排序
 - 快速插入、删除
 - 有序且支持范围查询
- 原生web开发跟springmvc区别
 - 就接受参数而言，Servlet需要使用requestParameter.get()
 - 而springmvc直接对我们前端传过来的参数进行封装成pojo
- filter跟springmvc中的interceptor区别
 - 拦截器：使用AOP的方式在请求handler前后新增加一些处理方法
 - filter: 在请求时过滤一些信息，或则提前设置一些参数，然后在交给servlet执行，如没有登录用户，不往下继续执行
- 了解的设计模式
- 单例模式实现方式

百度

- A、B、C系统分别为100QPS、200QPS、300QPS，如何设计一个限流的系统
 - <https://www.cnblogs.com/xuwc/p/9123078.html>
 - 漏桶算法，令牌算法
- 如何设计一个短连接
 - 基本原理，使用302 重定向（301 永久重定向）
 - marmurHash算法（随机分布特征更好），转换为64进制（节省长度），为了防止冲突，将原url + duplicate（自定义字符）重新计算hash，如果再次冲突则继续加duplicate，当计算原地址时，将duplicate去掉即可
 - 由于新增一条数据，需要查询、插入两次操作，容易出现性能瓶颈，可以将短连接增加一个唯一性索引
 - 高并发下也可以使用布隆过滤器，将生成的短链在布隆过滤器中校验
 - 也可以使用自增序列生成短链，如果DB压力过大，可以设置一个专门的发号表，每插入一条记录为短链id预留（主键id + 1000-9999）到（主键id * 1000）的号段
 - 为了防止多次相同的长连接生成不同的短连接，可以将长连接用md5适当压缩作为索引
- snowflake
 - 时间回拨，解决方案：
 1. 采用百度UIDGenerator，基于snowflake，采用双buffer来解决时钟回拨
 2. 美团 Leaf-Snowflake

- (1).mysql首先为了各种索引（覆盖索引、最左匹配），问的比较细致，然后给你个场景，让你判断走没走索引。
- (2).JVM说下G1垃圾回收器的、什么时候会STW、JVM哪块内存区域不会OOM。
- (3).IO模型说下BIO、AIO，阻塞到底是在哪个步骤阻塞，问下java中Epoll底层是LT还是ET。
- (4).说下HTTPS
- (5).synchronized、wait、notifyAll的底层实现，可重入锁、AQS
- (6).HashMap、HashTable、HashSet、ConcurrentHashMap
- (7).两个正序数组、寻找第K个小的数字（LeetCode寻找中位数的变形）Ologn复杂度

2.字节跳动提前批二面（7.21）

- (1).mysql（B+树索引，B树索引，Hash索引），explain关键字
- (2).操作系统的集中的线程模型，Java线程的中的线程机制（LWP），用户态访问内核态的方法
- (3).计算机网络（五层的作用，数据链路层的协议（PPP），网路层的（RIP，OSPF，BGP），传输层的（TCP,NAGLE协议），UDP），应用层写意思
- (4).JAVA final的关键字的作用（可以修饰哪些作用），集合框架（ArrayList,LinkedList,HashMap），反射的作用
- (5).分布式了解的作用（RPC）
- (6).算法（正序数组旋转，然后找到数组中的最小值（Ologn））

TCP采用大端？

- TCP/IP协议规定，网络数据应该采用大端的字节序，即低地址高字节

hashMap 为什么树化因子是8，而退化为链表是6

- 1 如果不设退化阈值，只以8来树化与退化：
- 2 那么8将成为一个临界值，时而树化，时而退化，此时会非常影响性能，因此，我们需要一个比8小的退化阈值；
- 3
- 4 UNTREEIFY_THRESHOLD = 7
- 5 同样，与上面的情况没有好多少，仅相差1个元素，仍旧会在链表与树之间反复转化；
- 6
- 7 那为什么是6呢？
- 8 源码中也说了，考虑到内存（树节点比普通节点内存大2倍，以及避免反复转化），所以，退化阈值最多为6。
- 9
- 10 避免链表和红黑树之间频繁的转换。如果阈值是7的话，删除一个元素红黑树就必须退化为链表，增加一个元素就必须树化，来回不断的转换结构无疑会降低性能，所以阈值才不设置的那么临界

小公司：

线程状态变化

sleep为啥不会释放锁

- 1 sleep可以在不持有对象锁的情况下执行，而wait方法必须在同步代码块中才能执行
- 2
- 3 sleep是Thread的方法，wait、notify都是Object中的方法，只要在同步代码块中才能调用

斯迈特：

js: intervalTime, timeout, js绑定事件

```

1    document.getElementById("btn2").onclick = show;
2    document.getElementById("btn3").addEventListener("click", show);
3
4    //移除事件监听
5    document.getElementById("btn3").removeEventListener("click");

```

vue生命周期、v-if、v-show、模板

行内元素、块内元素

js undefined跟null区别

```

1    * undefined代表定义未赋值
2    * null定义并赋值了,赋值为null

```

JSP内置对象、

输入url过程

```

1    本地域名服务器----->    根域名、顶级域名、权威域名、
2    .....

```

重写、重载、向上转型是调用的父类还是子类

```

1    重写, 权限修饰符子类的范围大于父类, 如父类为: protected, 子类不能为private, 答反了
2
3    调用子类, 多态的体现, 子类扩展的方法无法调用

```

synchronized跟ReentrantLock区别

```

1    1. 底层实现
2    2. 是否可手动释放
3    3. 是否可中断
4    4. 是否公平
5    5. 锁是否可以绑定Condition
6    6. 锁的对象,    Synchronized锁对象, 而ReentrantLock锁的线程

```

wait跟sleep区别

```

1    1、sleep是线程中的方法,但是wait是Object中的方法。
2    2、sleep方法不会释放lock,但是wait会释放,而且会加入到等待队列中。
3    3、sleep方法不依赖于同步器synchronized,但是wait需要依赖synchronized关键字。
4    4、sleep不需要被唤醒(休眠之后推出阻塞),但是wait需要(不指定时间需要被别人中断)。

```

JDK动态代理、CGLIB动态代理区别

```

1    JDK: 利用拦截器(拦截器必须实现InvocationHanlder)加上反射机制生成一个实现代理接口的匿名类,
2        在调用具体方法前调用InvokeHandler来处理
3        针对接口、利用反射机制
4        使用方式:
5            1、实现InvocationHandler
6            2、使用Proxy.newProxyInstance产生代理对象
7            3、被代理的对象必须要实现接口
8    CGLIB: 利用ASM开源包,对代理对象类的class文件加载进来,通过修改其字节码生成子类来处理。
9        使用继承委托类,重写委托类的方式,因此不能使用final修饰类或方法
10       使用方式:
11           1. 引入cglib相关的包
12           2. 实现MethodInterceptor

```

- 13 3. 实现方法，编写过滤规则
- 14 4. 创建Enhancer对象
- 15 由于反射的效率比较低，所以 CGlib 采用了FastClass 的机制来实现对被拦截方法的调用，大概就是对方法建立索引，使用switch调用。

过滤器适用场景

- 1 防止未登录就进入界面
- 2 控制应用编码
- 3 过滤敏感词汇等场景

数组转List? Arrays.asList(), 这种方式跟直接new ArrayList区别?

- 1 使用的Arrays的静态内部类ArrayList， 没有重写add, remove方法，抛异常
 UnsupportedOperationException

ArrayList遍历、删除一个指定的字符串，是否出现错误、forEach

```
1     // 报并发修改错误，实际底层就是Iterator在遍历，编译后可以查看， 从后往前不会报错
2     for (Integer integer : list) {
3         if (integer.equals(1)) {
4             list.remove(integer);
5         }
6     }
```

springmvc执行流程

spring如果解决多线程安全问题

- 1 多例
- 2 ThreadLocal

优点、缺点

转转面经:

为什么hashmap不使用AVL树，而使用红黑树

Redis 如何保证高可用的，获取一个数据的流程

线程池原理，初始线程是如何阻塞的

- 1 getTask()方法中，会通过allowCoreThreadTimeOut来判断，从workQueue获取任务，是调用take()还是poll(time),

JVM 年轻代与老年代的不同，常用的垃圾收集 算法， 标记清除 算法 用在年轻代可以吗

求给定无序数组的第k个大的元素（小顶堆，保证堆中元素为k， 部分快排）

自我介绍 + 项目

项目 具体负责什么

MySQL的索引优化

MySQL字段类型、范围以及存储长度的具体细节

MySQL的InnoDB和MyISAM两大引擎的区别以及适用场景

是否支持行级锁、是否支持事务、是否支持数据崩溃后恢复、是否支持MVCC

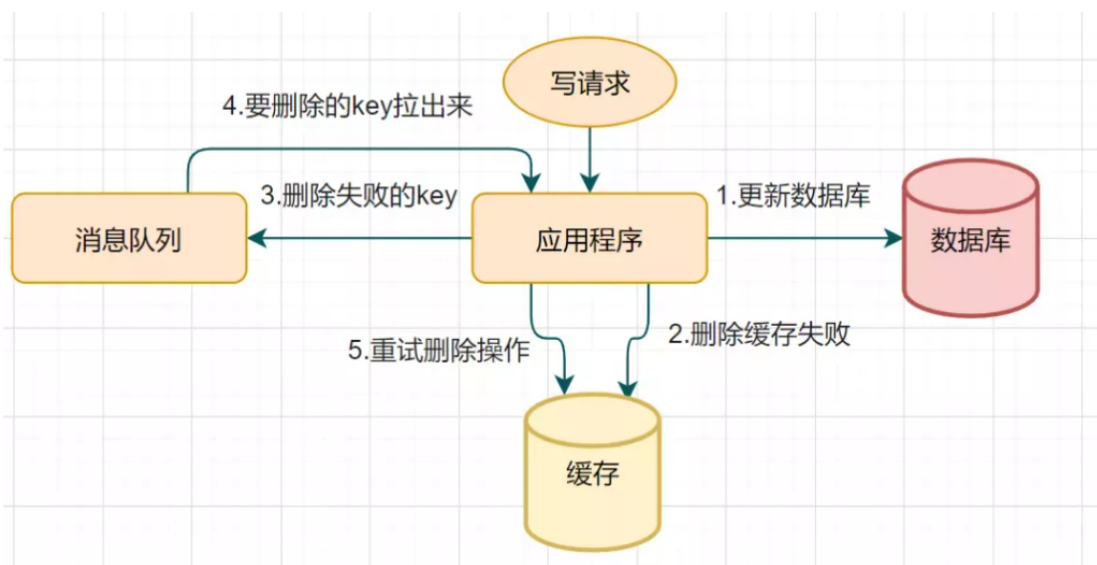
项目 中用到了Redis，怎么解决Redis缓存和数据库不一致的问题

1. 延时双删策略

- 1) 先删除缓存
- 2) 再写数据库
- 3) 休眠500毫秒，保证读请求已经完成
- 4) 再次删除缓存，可能会删除失败

2. 删除缓存重试机制

由于延时双删第二步可能会导致删除失败



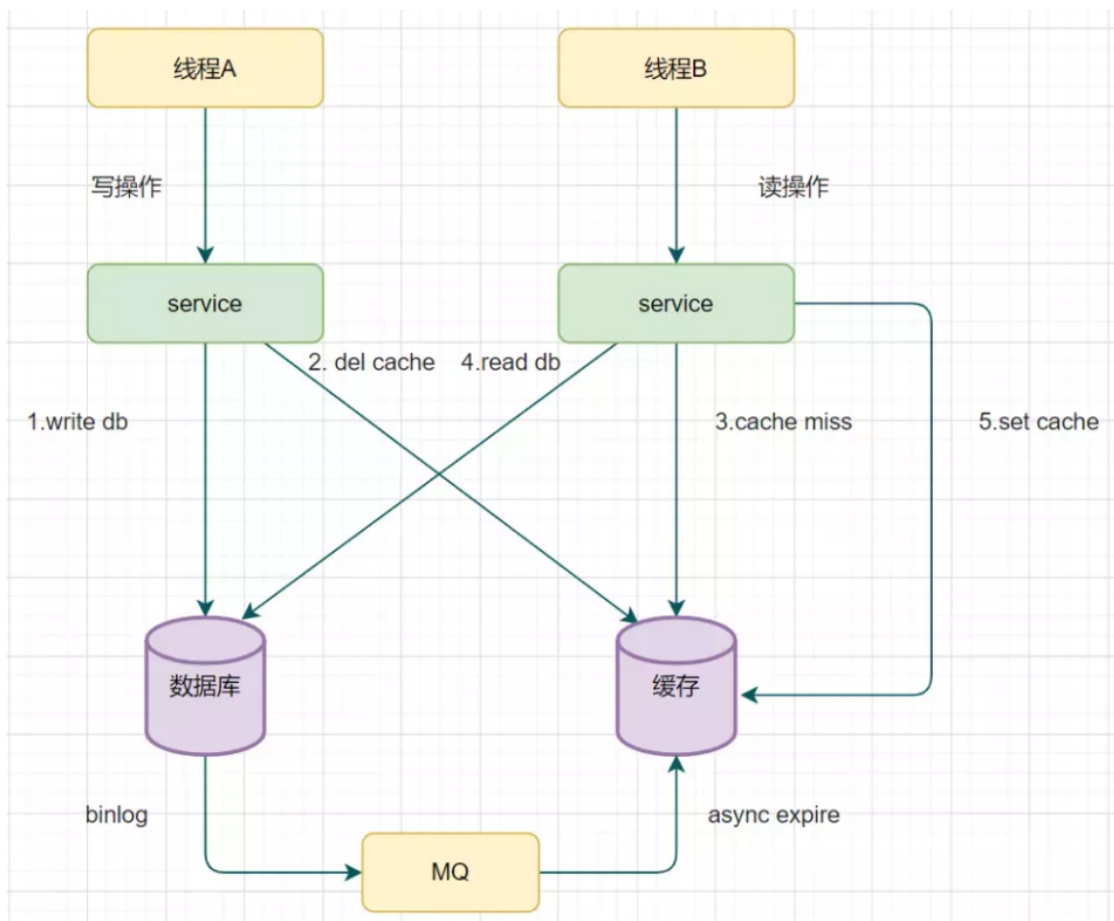
1. 写请求更新数据库
2. 缓存因为某些原因，删除失败
3. 把删除失败的key放到消息队列
4. 消费消息队列的消息，获取要删除的key
5. 重试删除缓存操作

3. 异步更新缓存（基于订阅同步binlog）

以mysql为例吧

可以使用阿里的canal将binlog日志采集发送到MQ队列里面

然后通过ACK机制确认处理这条更新消息，删除缓存，保证数据缓存一致性



项目 中有没有用到多线程，如何使用的

Netty中使用了多线程，8个workBoss？

项目 中为什么不采用线程池

线程池的底层实现（不需要讲参数之类的，要具体的实现逻辑）

Java中如何解决线程同步问题

Synchronized和Lock的异同

算法题：如何找出一组无序数组中最大的K个数，并优化

作者：为你永远铭记#

链接：https://www.nowcoder.com/discuss/730780?type=all&order=time&pos=&page=1&ncTraceId=&channel=-1&source_id=search_all_nctrack

来源：牛客网

自我介绍 + **项目**

Redis的持久化怎么实现

项目 的并发量有多少以及采用Redis的作用是什么

红黑树 了解吗？怎么实现的

HashMap的底层实现

场景题：对于慢SQL如何优化（分析原因、如何解决）

原因：

偶尔慢：

- redolog写满需要写入磁盘、内存不够需要淘汰lru、数据被加锁

一直慢：

- 字段没用索引、有索引由于SQL没用使用上where id = 100 + 1、函数操作导致索引失效、
- 如果索引区分度高就选择使用索引否则使用全部扫描，索引上不同的值越多区分度（基数）就越高，通过采样遍历部分数据来预测索引的基数

场景题：对于一个数量巨大的订单表，如何采用分库分表（优化思路和实现细节）

AOP中为什么无法拦截内部方法调用

Java中有哪些锁

Synchronized和Lock的异同

项目中的收获

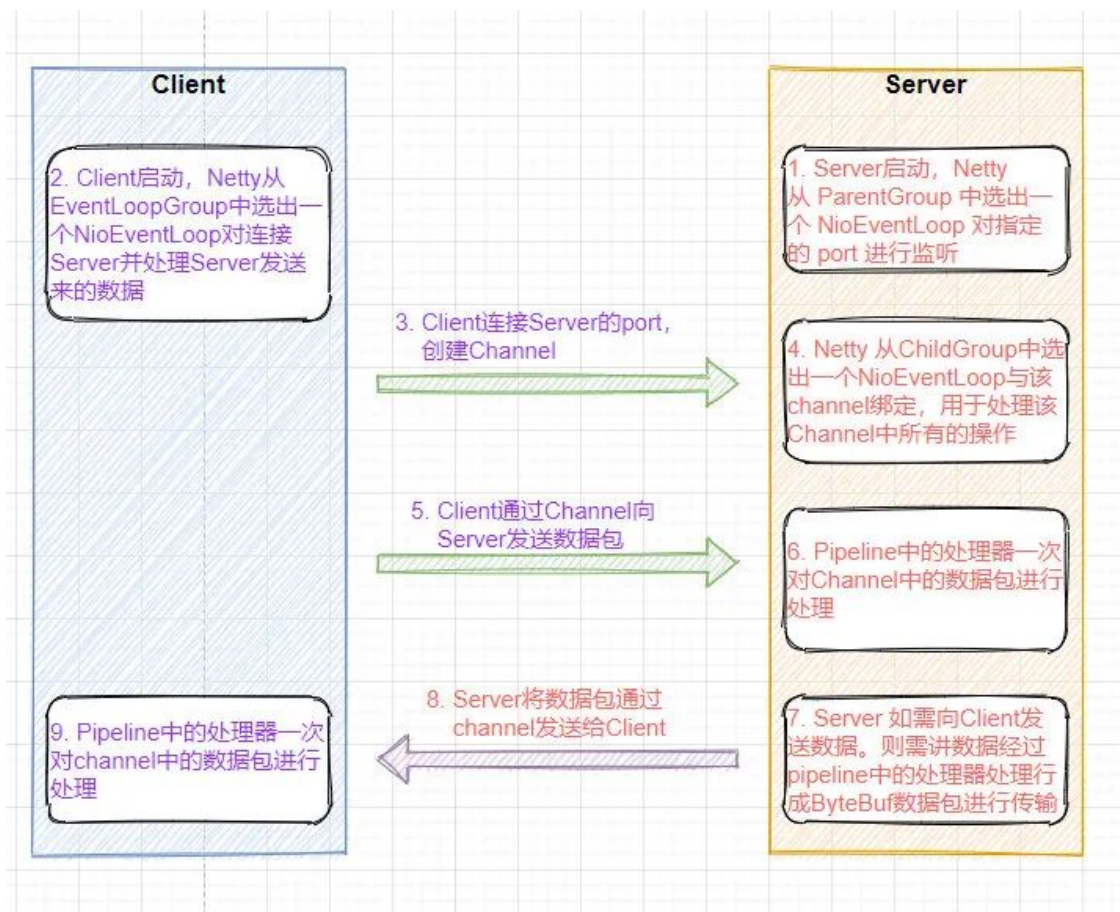
个人的优点是什么

个人的缺点是什么

自己一面结束流程：

1. 介绍
2. 项目，由于用了Netty，说为什么用Netty，Netty优点，Netty组件，怎么分配内存

```
1 Channel NioServerSocketChannel NioSocketChannel
2 EventLoop
3 EventLoopFuture 异步非阻塞
4 ChannelHandler
5 ChannelPipeline
6 Bootstrap serverBootstrap
```



3. 回收算法, CMS, G1, 详细说下G1的复制过程(是否需要标记), 总结他们优缺点, G1是否解决了CMS的一些缺点(内存碎片, 浮动垃圾? 🤔)

- 1 CMS: 初始标记(STW)、并发标记(采用增量更新记录发生变化的对象)、重新标记(STW)、并发清除
- 2 在并发标记阶段、并发清除阶段都会产生浮动垃圾
- 3
- 4 G1: 面向Region, 可以对堆内存中任何一部分组成回收集(CSet)进行回收
- 5 Minor GC: 根扫描、更新&处理Rest、复制对象
- 6 对于年轻代Region来说, Rset只保留老年代对年轻代的引用,
- 7 对于老年代的Region来说, RSet也只会保存老年代对他的引用, 老年代回收之前会对年轻代进行回收一次(是不是minor GC时, 如果老年代引用了年轻代, 那么GC ROOTS中已经有老年代的对象, 在老年代回收时, 对于年轻代指向老年代的对象已经在GC ROOTS中, 因此就不需要在保存年轻代对老年代的引用)
- 8
- 9 Mixed GC: 当堆空间占有率达到45%时, 在触发Mixed GC时一定会回收年轻代, 采集部分老年代收集
- 10 初始标记(STW)、并发标记(使用SATB记录引用发生变化的对象)、最终标记(STW)、筛选清除(STW, 涉及到复制对象)
- 11 SATB有个问题, 如果开始G1认为对象是活的, 那么在此次GC中不会被回收, 即便他已经变为了垃圾, 因此依然会有浮动垃圾产生, 相当于SATB只记录第一步标记为垃圾的对象复活了?
- 12
- 13 CMS优点: 减少用户线程停顿时间, 垃圾回收器的内存占用没有G1那么大
- 14 缺点: 采用标记清除算法, 容易产生大量的内存碎片, 导致分配大对象可能找不到合适的区域从而触发Full GC
- 15
- 16 G1优点: 面向Region, 提供可预测的停顿时间, 使用复制算法, 解决空间碎片的问题, 采用SATB的方式记录发生变化的对象, 相比增量更新算法, 原始快照搜索能够减少并发标记和重新标记阶段的消耗, 避免CMS那样在最终标记阶段停顿时间过长的缺点。
- 17 缺点: 相比CMS来说, G1需要为每一个Region都维护一个记录集(RSet), 以致于垃圾回收线程占用的内存更高
- 18

19	都有浮动垃圾的产生
20	Guide
21	CMS
22	对 CPU 资源敏感;
23	无法处理浮动垃圾;
24	它使用的回收算法-“标记-清除”算法会导致收集结束时会有大量空间碎片产生

4. 写算法，知道树的先序、中序，算后续